Multiplataforma de Rutas de Transporte Urbano — PiuraGo

Avance de Proyecto Final 2 — Semana 08

Leyter Harvey Alama Cuello

Yolanda Isabel Castro Ortega

Mia Fernanda Cordova Peña

Iván Alfonso Quezada Girón

Ivan Zapata Cornejo

Escuela de Ingeniería de Sistemas e Informática, Facultad de Ingeniería

Universidad Tecnológica del Perú

Curso Integrador I: Sistemas Software

Ing. Darwin Jhoel Sullon Barranzuela

29 de septiembre de 2026

1. Índice

2. Introducción 4
2.1. Contexto del problema 4
2.2. Objetivos del proyecto 5
2.2.1. Objetivo general 5
2.2.2. Objetivos específicos 5
2.3. Alcance de la solución 6
2.3.1. Dentro del alcance 6
2.3.2. Fuera del alcance 6
3. Planificación y análisis 7
3.1. Lean Canvas del proyecto 7
3.2. Planificación con PMBOK 9
3.2.1. Project Charter 9
3.2.2. Diagrama de Gantt 12
3.2.3. Work Breakdown Structure (WBS) 13
3.3. Toma de requerimientos 17
3.3.1. Requerimientos funcionales 17
3.3.2. Requerimientos no funcionales 18
3.3.3. Clasificación DINO / MoSCoW 19
4. Modelamiento de procesos de negocio – BPM 19
4.1. Importancia del modelamiento de procesos 19
4.2. Diagramas BPMN del proyecto 20
4.2.1. Diagrama del proceso principal 20
4.2.2. Diagramas de procesos secundarios 21
5. Diseño de base de datos 25
5.1. Diseño lógico de la base de datos 25

5.1.1. Modelo entidad-relación 25
5.2. Diseño físico de la base de datos 26
6. Diseño UX/UI y prototipado 28
6.1. Identificación de usuarios y necesidades 28
6.2. Diseño UX 29
6.2.1. Diseño de las experiencias principales 29
6.3. Diseño UI y prototipado 30
6.3.1. Prototipo de la solución 30
6.3.2. Herramienta utilizada para el prototipado 30
6.3.3. Pantallas de los procesos principales 31
7. Reportes del sistema 37
7.1. Identificación de reportes 37
7.1.1. Reportes requeridos por el proyecto 37
8. Referencias 39

Multiplataforma de Rutas de Transporte Urbano — PiuraGo

Introducción

Contexto del problema

En Piura, el transporte urbano opera principalmente a través de combis y colectivos gestionados por empresas privadas informales, sin un sistema centralizado que informe a los usuarios sobre la ubicación de las unidades, el tiempo estimado de llegada a un paradero o las franjas horarias y zonas de mayor congestión. Las rutas frecuentadas por estudiantes y trabajadores que se desplazan hacia zonas como el Centro de Piura, la Universidad Nacional de Piura, la UDEP o los principales centros comerciales presentan alta variabilidad en los tiempos de espera, agravada por el tráfico en horas punta y por la falta de información pública sobre horarios y recorridos.

Esta incertidumbre genera pérdida de tiempo, dificultad para planificar desplazamientos y una percepción de baja confiabilidad del servicio. A diferencia de ciudades con sistemas de transporte formal, como el Metropolitano en Lima, Piura carece de una plataforma digital que centralice información de rutas, paraderos y tiempos de llegada para el transporte urbano existente.

El público objetivo son los usuarios cotidianos del transporte urbano en Piura, principalmente estudiantes universitarios y trabajadores, que actualmente dependen de la experiencia previa, de recomendaciones informales o simplemente de esperar en el paradero sin ninguna referencia de tiempo. Dado que las empresas de combis no cuentan con GPS instalado en sus unidades ni existe convenio con el equipo para acceder a esa información, la solución adopta un enfoque colaborativo: aprovechar el GPS de los propios pasajeros a bordo para estimar la posición de las unidades, calcular tiempos de llegada y detectar tramos y horarios de mayor congestión, sin depender de hardware instalado en las combis.

Objetivos del proyecto

Objetivo general

Contribuir a mejorar la gestión del tráfico y el flujo del transporte urbano en Piura mediante una aplicación móvil multiplataforma que, a partir del GPS colaborativo de los propios pasajeros, brinde información sobre rutas, tiempos estimados de llegada y zonas u horarios de mayor congestión, permitiendo a los usuarios planificar mejor sus desplazamientos y distribuir la demanda de transporte de forma más informada.

Objetivos específicos

- Diseñar e implementar un módulo de consulta de rutas y paraderos que permita al usuario identificar las combis disponibles y sus puntos de parada en la ciudad de Piura.
- Desarrollar un servicio de cálculo de ETA basado en los reportes de ubicación GPS enviados por pasajeros colaboradores, aplicando una arquitectura MVC + DAO bajo principios SOLID, que ayude al usuario a evitar tiempos de espera innecesarios en los paraderos.
- Construir un mapa interactivo con la posición estimada de las unidades y un dashboard de zonas y franjas horarias de mayor congestión, que permita a los usuarios identificar patrones de tráfico y ajustar sus horarios o rutas de desplazamiento.

Alcance de la solución

El alcance del proyecto se delimita explícitamente para que el equipo pueda completarlo dentro de las 18 semanas del curso, priorizando la demostración de la arquitectura MVC + DAO + SOLID + TDD, el modelo de datos y el flujo funcional completo, sin depender de infraestructura externa fuera de control del equipo.

Dentro del alcance

- Registro y autenticación básica de usuarios.
- Consulta de rutas disponibles y sus paraderos asociados, con nombre, ubicación y orden secuencial.
- Captura del GPS del pasajero mientras viaja en una unidad, para reportar su posición como colaborador activo de esa ruta.
- Cálculo de ETA de una unidad hacia un paradero seleccionado, en base a los reportes de ubicación GPS más recientes enviados por los pasajeros colaboradores a bordo.
- Mapa interactivo con la posición estimada de las unidades, actualizada mediante polling HTTP cada 15–20 segundos.
- Dashboard de "horas calientes": franjas horarias y tramos de ruta con mayor congestión, calculado sobre el histórico de reportes GPS.
- Notificación tipo geofencing: aviso al usuario cuando su unidad está cerca de su paradero.

Fuera del alcance

- Cobertura garantizada en rutas sin colaboradores activos: la precisión del ETA y del mapa depende de que haya pasajeros compartiendo su ubicación en ese momento.
- Comunicación en tiempo real verdadero vía WebSockets: se reemplaza por polling HTTP, que cumple el mismo objetivo visual con menor complejidad de infraestructura.
- Verificación de identidad o validación de que el colaborador efectivamente se encuentra a bordo de la unidad reportada.
- Gestión administrativa para las empresas de transporte.

Planificación y análisis

Lean Canvas del proyecto

A continuación se describe cada componente del Lean Canvas elaborado por el equipo para el proyecto de rutas de transporte urbano en Piura (Autor del canal, s. f.). La imagen del canvas completo fue elaborada en Canva y se muestra en la Figura 1.

Figura 1

Lean Canvas del proyecto PiuraGo



Nota. Elaboración propia, realizada en Canva.

El problema identificado es la falta de información confiable sobre rutas, la ausencia de ubicación en tiempo real y el desorden del transporte por falta de paradas obligatorias. El segmento de clientes son estudiantes universitarios y trabajadores de 15 a 50 años que usan transporte público en Piura. La propuesta de valor única es ofrecer visibilidad en tiempo real de las combis y colectivos impulsada por la misma comunidad de pasajeros, sin depender de hardware instalado en las unidades. La solución consiste en una aplicación multiplataforma colaborativa que utiliza el

GPS de los propios pasajeros para mapear la trayectoria, las paradas habilitadas y la posición aproximada de las unidades, calculando tiempos estimados de llegada y detectando puntos y horarios críticos de tráfico.

Los canales de distribución previstos son un enlace web y APK durante la fase piloto, con difusión en redes sociales universitarias y grupos de estudiantes de la zona. Como fuente de ingresos, al tratarse de un proyecto académico sin fines de lucro, se plantea como hipótesis de escalamiento futuro un modelo freemium o de publicidad local de comercios cercanos a los paraderos. La estructura de costos se limita al hosting del backend y la base de datos en planes gratuitos (Render/Railway/Neon), al tiempo de desarrollo del equipo y al uso de herramientas open source (Flutter, flutter_map, Spring Boot), sin costos de licencias. La ventaja competitiva es contar con un modelo de rastreo colaborativo completamente independiente de las empresas de transporte informal, impulsado por una masa inicial de usuarios activos. Finalmente, las métricas clave incluyen el número de consultas de ETA realizadas, el porcentaje de usuarios que comparten su ubicación en tiempo real, la cobertura de rutas con al menos un colaborador activo, el margen de error promedio del ETA calculado, el tiempo promedio de espera reportado por paradero, y los tramos u horarios con mayor congestión detectada.

Planificación con PMBOK

Project Charter

De acuerdo con la guía del PMBOK, el Project Charter es el documento que autoriza formalmente la existencia del proyecto y otorga al equipo la autoridad para aplicar los recursos disponibles a sus actividades (Cañabi Educa, s. f.; Experiencias de un Maestro, s. f.). A continuación se presenta el acta de constitución del proyecto PiuraGo.

Nombre del proyecto. Multiplataforma de Rutas de Transporte Urbano — Piura PiuraGo

Patrocinador. Este proyecto es de carácter educativo, en representación de la Escuela de Ingeniería de Sistemas e Informática de la Universidad Tecnológica del Perú.

Equipo responsable del proyecto. Alama Cuello, Leyter Harvey; Castro Ortega, Yolanda Isabel; Cordova Peña, Mia Fernanda; Quezada Girón, Iván Alfonso; y Zapata Cornejo, Ivan, estudiantes del curso e integrantes con responsabilidad compartida sobre la planificación, el desarrollo y la sustentación del proyecto.

Propósito y justificación. No existe en Piura una herramienta digital centralizada para el transporte urbano informal. El proyecto responde a una necesidad real de la comunidad universitaria —estudiantes y trabajadores que dependen de combis y colectivos sin información confiable de rutas, paraderos ni tiempos de llegada— y, a la vez, permite al equipo aplicar de forma integrada las competencias exigidas por el sílabo del curso: planificación, diseño, desarrollo, pruebas, despliegue y mantenimiento de software.

Objetivo general del proyecto. Construir una solución informática multiplataforma que contribuya a la gestión del tráfico y el transporte urbano en Piura, permitiendo consultar rutas, calcular el tiempo estimado de llegada y visualizar horas y tramos de mayor congestión a partir del GPS colaborativo de los propios pasajeros, siguiendo la arquitectura MVC + DAO + SOLID + TDD exigida por el sílabo.

Descripción de alto nivel y entregables principales. El proyecto se organiza en cuatro entregables sucesivos, alineados a las cuatro unidades del curso: (1) documento de planificación y análisis; (2) documento de diseño y arquitectura; (3) avance funcional del backend y del cliente móvil (arquitectura MVC + DAO, módulo de rutas/GPS y servicio de ETA); y (4) el sistema completo, probado, desplegado y con planes de monitoreo y mantenimiento.

Criterios de éxito. Se considerará exitoso el proyecto si, al finalizar las 18 semanas, el sistema permite calcular y mostrar el ETA de al menos una ruta piloto con datos de colaboradores reales o simulados, el backend cumple la arquitectura MVC + DAO + SOLID exigida y cuenta con pruebas automatizadas, y el equipo sustenta la solución ante el docente cumpliendo todos los entregables del sílabo.

Hitos principales. Semana 4: entrega de APF1 (20%). Semana 8: entrega de APF2 (20%). Semana 12: entrega de APF3 (20%). Semana 18: entrega y sustentación del proyecto final (40%).

Presupuesto y recursos estimados. El proyecto se desarrolla bajo una restricción explícita de costo $0, al tratarse de un proyecto académico sin fines de lucro. Todos los servicios utilizados cuentan con un plan gratuito suficiente para el alcance del piloto; el único recurso invertido es el tiempo de desarrollo del propio equipo.

Supuestos. Se asume que un número suficiente de pasajeros aceptará compartir su ubicación GPS como colaboradores activos durante el piloto; que las unidades cuentan con cobertura de datos móviles suficiente para el envío periódico de ubicación; y que los colaboradores reportan su posición de buena fe.

Restricciones. Sin GPS instalado en las combis, por lo que se depende del GPS de los pasajeros colaboradores; sin comunicación en tiempo real verdadero vía WebSockets, sustituida por polling HTTP; desarrollo limitado a herramientas con plan gratuito; y alcance acotado a las 18 semanas del curso.

Riesgos iniciales. (1) Retrasos por sobrecarga académica del ciclo; (2) baja adopción inicial de colaboradores activos, lo que reduce la cobertura y precisión del ETA en el piloto, mitigado enfocando el lanzamiento en una comunidad universitaria concentrada; (3) curva de aprendizaje del stack elegido; (4) dependencia de servicios gratuitos que pueden tener límites de uso; (5)

reportes de ubicación falsos o inconsistentes al no validarse que el colaborador esté físicamente a bordo de la unidad.

Stakeholders. Equipo de desarrollo (estudiantes), docente del curso (patrocinador y evaluador), usuarios finales —estudiantes y trabajadores que usan transporte urbano, tanto colaboradores que comparten su GPS como consumidores de la información— y, potencialmente, empresas de combis en fases futuras.

Autoridad del equipo del proyecto. El equipo de desarrollo tiene autoridad para tomar las decisiones técnicas de arquitectura, selección de herramientas y priorización de requerimientos dentro del alcance definido en este documento, sujeto a la revisión y aprobación del docente del curso en cada hito de entrega.

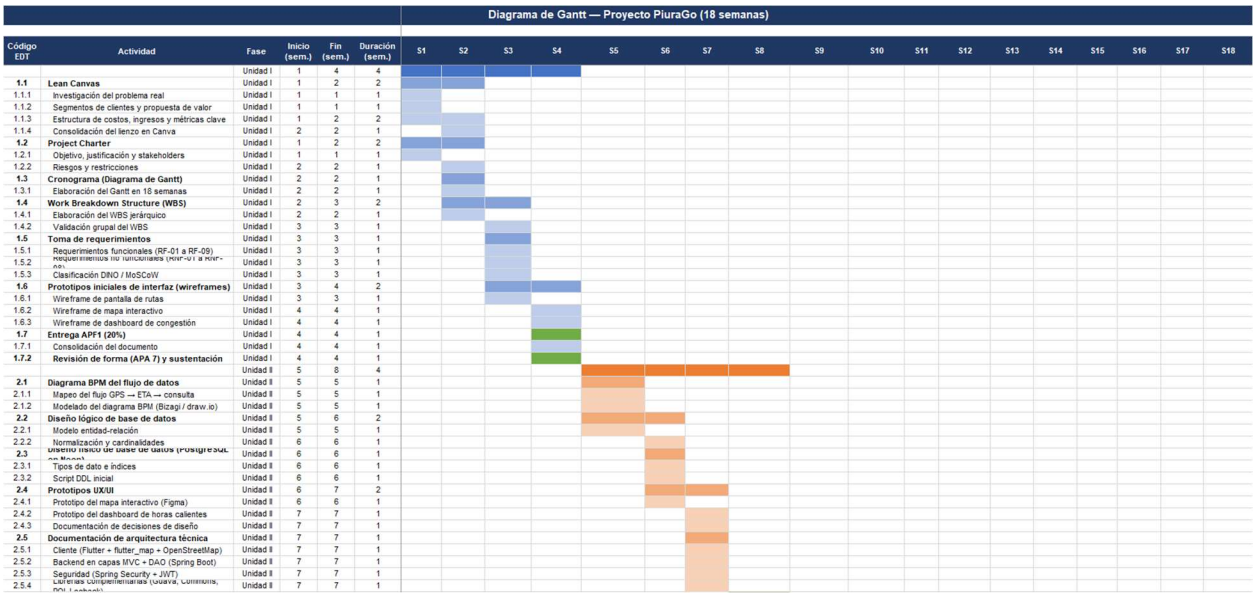
Aprobación. Este Project Charter se considera aprobado con la evaluación favorable del presente Avance de Proyecto Final 1 por parte del docente del curso, Ing. Luis A. Zapata Mantilla.

Diagrama de Gantt

El cronograma del proyecto se alinea a las entregas del sílabo del curso, distribuidas en las cuatro unidades de aprendizaje (Antonio Cedillo Hernandez, s. f.). Las Figuras 2 y 3 muestran la versión vigente del diagrama de Gantt; este diagrama será actualizado manualmente por el equipo conforme avance el proyecto.

Figura 2

Diagrama de Gantt del proyecto PiuraGo: Unidades I y II

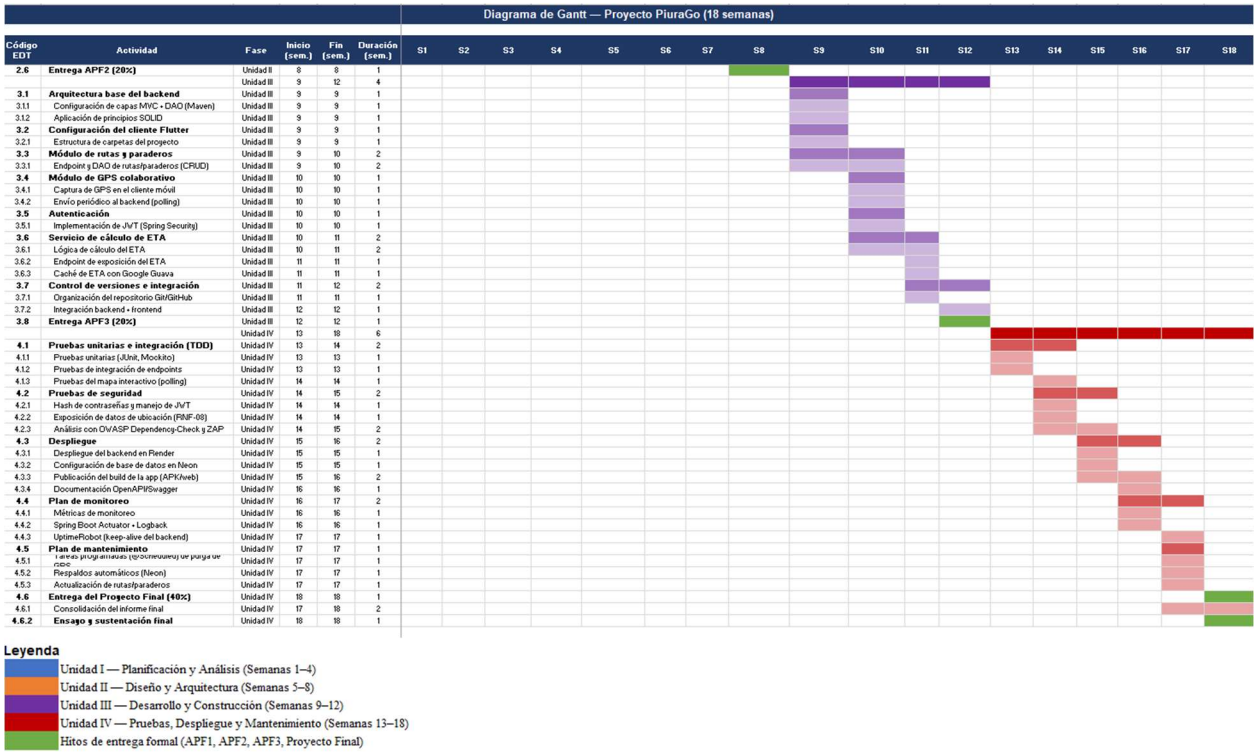


Código EDT	Actividad	Fase	Inicio (sem.)	Fin (sem.)	Duración (sem.)
		Unidad I	1	4	4
1.1	**Lean Canvas**	Unidad I	1	2	2
1.1.1	Investigación del problema real	Unidad I	1	1	1
1.1.2	Segmentos de clientes y propuesta de valor	Unidad I	1	1	1
1.1.3	Estructura de costos, ingresos y métricas clave	Unidad I	1	2	2
1.1.4	Consolidación del lienzo en Canva	Unidad I	2	2	1
1.2	**Project Charter**	Unidad I	1	2	2
1.2.1	Objetivo, justificación y stakeholders	Unidad I	1	1	1
1.2.2	Riesgos y restricciones	Unidad I	2	2	1
1.3	**Cronograma (Diagrama de Gantt)**	Unidad I	2	2	1
1.3.1	Elaboración del Gantt en 18 semanas	Unidad I	2	2	1
1.4	**Work Breakdown Structure (WBS)**	Unidad I	2	3	2
1.4.1	Elaboración del WBS jerárquico	Unidad I	2	2	1
1.4.2	Validación grupal del WBS	Unidad I	3	3	1
1.5	**Toma de requerimientos**	Unidad I	3	3	1
1.5.1	Requerimientos funcionales (RF-01 a RF-09)	Unidad I	3	3	1
1.5.2	Requerimientos no funcionales (RNF-01 a RNF-06)	Unidad I	3	3	1
1.5.3	Clasificación DINO / MoSCoW	Unidad I	3	3	1
1.6	**Prototipos iniciales de interfaz (wireframes)**	Unidad I	3	4	2
1.6.1	Wireframe de pantalla de rutas	Unidad I	3	3	1
1.6.2	Wireframe de mapa interactivo	Unidad I	4	4	1
1.6.3	Wireframe de dashboard de congestión	Unidad I	4	4	1
1.7	**Entrega APF1 (20%)**	Unidad I	4	4	1
1.7.1	Consolidación del documento	Unidad I	4	4	1
1.7.2	**Revisión de forma (APA 7) y sustentación**	Unidad I	4	4	1
		Unidad II	5	8	4
2.1	**Diagrama BPM del flujo de datos**	Unidad II	5	5	1
2.1.1	Mapeo del flujo GPS → ETA → consulta	Unidad II	5	5	1
2.1.2	Modelado del diagrama BPM (Bizagi / draw.io)	Unidad II	5	5	1
2.2	**Diseño lógico de base de datos**	Unidad II	5	6	2
2.2.1	Modelo entidad-relación	Unidad II	5	5	1
2.2.2	Normalización y cardinalidades	Unidad II	6	6	1
2.3	**Diseño físico de base de datos (PostgreSQL on Neon)**	Unidad II	6	6	1
2.3.1	Tipos de dato e índices	Unidad II	6	6	1
2.3.2	Script DDL inicial	Unidad II	6	6	1
2.4	**Prototipos UX/UI**	Unidad II	6	7	2
2.4.1	Prototipo del mapa interactivo (Figma)	Unidad II	6	6	1
2.4.2	Prototipo del dashboard de horas calientes	Unidad II	7	7	1
2.4.3	Documentación de decisiones de diseño	Unidad II	7	7	1
2.5	**Documentación de arquitectura técnica**	Unidad II	7	7	1
2.5.1	Cliente (Flutter + flutter_map + OpenStreetMap)	Unidad II	7	7	1
2.5.2	Backend en capas MVC + DAO (Spring Boot)	Unidad II	7	7	1
2.5.3	Seguridad (Spring Security + JWT)	Unidad II	7	7	1
2.5.4	Librerías complementarias (Guava, Commons, DQL Lombok)	Unidad II	7	7	1

Nota. Elaboración propia. El diagrama está alineado a las cuatro unidades del sílabo.

Figura 3

Diagrama de Gantt del proyecto PiuraGo: Unidades II, III y IV



Nota. Elaboración propia. La leyenda inferior indica los colores de cada unidad y los hitos de entrega.

Work Breakdown Structure (WBS)

La Estructura de Desglose del Trabajo descompone el proyecto en cuatro fases principales (lepatru007, s. f.). Cada fase se divide en paquetes de trabajo (y estos, a su vez, en actividades concretas y verificables), de modo que cada entregable del proyecto pueda rastrearse hasta una tarea específica y a un responsable dentro del cronograma. Esta descomposición jerárquica, presentada en la Tabla 1, constituye la línea base de alcance del proyecto y es la referencia principal para la planificación del cronograma detallado por semana.

Tabla 1

Estructura de desglose del trabajo (WBS) del proyecto PiuraGo

Código EDT	Paquete de trabajo / actividad	Descripción / entregable
1.0	***Planificación y Análisis (Unidad I — Semanas 1 a 4)***	*Entregable: APF1 (20%). Conceptualización del proyecto, levantamiento de requerimientos y línea base de planificación.*
1.1	**Lean Canvas**	Modelo de negocio del proyecto.
1.1.1	Investigación del problema real	Indagar cómo funciona hoy la distribución y el acceso a paradas y rutas de las combis en Piura.
1.1.2	Segmentos de clientes y propuesta de valor	Definir el público objetivo (estudiantes y trabajadores) y la propuesta de valor única de la plataforma.
1.1.3	Estructura de costos, ingresos y métricas clave	Establecer costos de hosting/desarrollo, hipótesis de ingresos futuros y KPIs de seguimiento.
1.1.4	Consolidación del lienzo en Canva	Armar y revisar en equipo el Lean Canvas final.
1.2	**Project Charter**	Acta de constitución del proyecto.
1.2.1	Objetivo, justificación y stakeholders	Redactar el objetivo del proyecto, su justificación académica/social y el mapa de interesados.
1.2.2	Riesgos y restricciones	Levantar los riesgos iniciales (adopción, curva de aprendizaje, servicios gratuitos) y las restricciones declaradas del proyecto.
1.3	**Cronograma (Diagrama de Gantt)**	Planificación temporal del proyecto.
1.3.1	Elaboración del Gantt en 18 semanas	Construir el cronograma alineado a las cuatro unidades del sílabo del curso.
1.4	**Work Breakdown Structure (WBS)**	Descomposición jerárquica del trabajo.
1.4.1	Elaboración del WBS jerárquico	Descomponer cada fase en paquetes de trabajo y actividades verificables.
1.4.2	Validación grupal del WBS	Revisión cruzada del desglose entre los integrantes del equipo.
1.5	**Toma de requerimientos**	Especificación funcional y no funcional del sistema.
1.5.1	Requerimientos funcionales (RF-01 a RF-09)	Redactar los requerimientos funcionales del sistema.
1.5.2	Requerimientos no funcionales (RNF-01 a RNF-08)	Redactar los requerimientos no funcionales del sistema.
1.5.3	Clasificación DINO / MoSCoW	Priorizar los requerimientos según su criticidad para el piloto académico.
1.6	**Prototipos iniciales de interfaz (wireframes)**	Primeras vistas de baja fidelidad de la aplicación.
1.6.1	Wireframe de pantalla de rutas	Boceto de la pantalla de consulta de rutas y paraderos.
1.6.2	Wireframe de mapa interactivo	Boceto de la pantalla del mapa con posición estimada de unidades.
1.6.3	Wireframe de dashboard de congestión	Boceto del panel de “horas calientes”.
1.7	**Entrega APF1**	Cierre formal de la fase de planificación.

Código EDT	Paquete de trabajo / actividad	Descripción / entregable
1.7.1	Consolidación del documento	Integrar Lean Canvas, Project Charter, Gantt, WBS, requerimientos y wireframes en un solo documento.
1.7.2	Revisión de forma y sustentación	Revisión ortográfica/formato en APA 7 y preparación de la exposición.
2.0	***Diseño y Arquitectura (Unidad II — Semanas 5 a 8)***	*Entregable: APF2 (20%). Modelado técnico y visual del sistema.*
2.1	**Diagrama BPM del flujo de datos**	Modelo del proceso de negocio extremo a extremo.
2.1.1	Mapeo del flujo GPS → ETA → consulta	Identificar y ordenar los pasos desde el reporte GPS hasta la consulta del usuario.
2.1.2	Modelado del diagrama BPM	Construir el diagrama en Bizagi o draw.io.
2.2	**Diseño lógico de base de datos**	Modelo entidad-relación del dominio.
2.2.1	Modelo entidad-relación	Modelar entidades: usuarios, rutas, paraderos y reportes GPS.
2.2.2	Normalización y cardinalidades	Revisar en equipo la normalización y las relaciones entre entidades.
2.3	**Diseño físico de base de datos (PostgreSQL en Neon)**	Implementación física del modelo de datos.
2.3.1	Tipos de dato e índices	Definir tipos de dato, claves e índices sobre las tablas principales.
2.3.2	Script DDL inicial	Redactar y validar el script de creación de tablas.
2.4	**Prototipos UX/UI**	Diseño de interfaz de alta fidelidad.
2.4.1	Prototipo del mapa interactivo	Prototipar en Figma la vista de mapa con flutter_map.
2.4.2	Prototipo del dashboard de "horas calientes"	Prototipar el panel de congestión por franja horaria.
2.4.3	Documentación de decisiones de diseño	Registrar paleta de colores, componentes y navegación definidos.
2.5	**Documentación de arquitectura técnica**	Sustento técnico de cada componente del sistema.
2.5.1	Cliente (Flutter + flutter_map + OpenStreetMap)	Documentar el rol del cliente móvil/web y la librería de mapas.
2.5.2	Backend en capas MVC + DAO (Spring Boot)	Documentar la organización en Controller, Service y DAO/Repository.
2.5.3	Seguridad (Spring Security + JWT)	Documentar el mecanismo de autenticación y autorización.
2.5.4	Librerías complementarias	Documentar el uso de Google Guava, Apache Commons, Apache POI y Logback.
2.6	**Entrega APF2**	Cierre formal de la fase de diseño.
3.0	***Desarrollo y Construcción (Unidad III — Semanas 9 a 12)***	*Entregable: APF3 (20%). Implementación del backend, el cliente y la lógica de negocio.*
3.1	**Arquitectura base del backend**	Configuración inicial del proyecto Spring Boot.
3.1.1	Configuración de capas MVC + DAO	Configurar el proyecto con Maven y la estructura de paquetes por capa.

Código EDT	Paquete de trabajo / actividad	Descripción / entregable
3.1.2	Aplicación de principios SOLID	Revisar en equipo la aplicación de SOLID en las clases base.
3.2	**Configuración del cliente Flutter**	Estructura base de la aplicación móvil/web.
3.2.1	Estructura de carpetas del proyecto	Configurar el proyecto Flutter y su organización interna.
3.3	**Módulo de rutas y paraderos**	CRUD de rutas y paraderos.
3.3.1	Endpoint y DAO de rutas/paraderos	Implementar el CRUD básico sobre rutas y paraderos.
3.4	**Módulo de GPS colaborativo**	Captura y envío de ubicación del pasajero.
3.4.1	Captura de GPS en el cliente móvil	Implementar la lectura de ubicación en Flutter.
3.4.2	Envío periódico al backend (polling)	Enviar la ubicación cada 5 a 10 segundos mediante peticiones POST.
3.5	**Autenticación**	Módulo de acceso seguro.
3.5.1	Implementación de JWT	Configurar Spring Security con emisión y validación de tokens JWT.
3.6	**Servicio de cálculo de ETA**	Lógica central del sistema.
3.6.1	Lógica de cálculo del ETA	Calcular el ETA a partir de los reportes GPS más recientes.
3.6.2	Endpoint de exposición del ETA	Exponer el ETA calculado por ruta y paradero.
3.6.3	Caché de ETA con Google Guava	Cachear en memoria el ETA reciente para evitar recálculos en cada polling.
3.7	**Control de versiones e integración**	Trabajo colaborativo sobre el repositorio.
3.7.1	Organización del repositorio Git/GitHub	Definir ramas, convenciones de commits y Pull Requests.
3.7.2	Integración backend + frontend	Integrar el avance del backend con el cliente Flutter y resolver bloqueos.
3.8	**Entrega APF3**	Cierre formal de la fase de desarrollo.
4.0	***Pruebas, Despliegue y Mantenimiento (Unidad IV — Semanas 13 a 18)***	*Entregable: Proyecto final (40%). Verificación, publicación y sostenibilidad del sistema.*
4.1	**Pruebas unitarias e integración (TDD)**	Verificación funcional del sistema.
4.1.1	Pruebas unitarias (JUnit, Mockito)	Probar el servicio de ETA y los DAOs simulando la capa de datos.
4.1.2	Pruebas de integración de endpoints	Probar en conjunto los endpoints de rutas, GPS y autenticación.
4.1.3	Pruebas del mapa interactivo	Verificar la actualización del mapa mediante polling cada 5 a 10 segundos.
4.2	**Pruebas de seguridad**	Verificación de la protección de datos y accesos.
4.2.1	Hash de contraseñas y manejo de JWT	Verificar el cifrado de contraseñas y la validez de los tokens.
4.2.2	Exposición de datos de ubicación (RNF-08)	Revisar la minimización de retención y el anonimato de los colaboradores.
4.2.3	Análisis con OWASP Dependency-Check y ZAP	Escanear dependencias vulnerables y probar la API desplegada.
4.3	**Despliegue**	Publicación del sistema en producción piloto.

Código EDT	Paquete de trabajo / actividad	Descripción / entregable
4.3.1	Despliegue del backend en Render	Publicar el.jar generado por Maven en el hosting gratuito.
4.3.2	Configuración de base de datos en Neon	Configurar la instancia de PostgreSQL en producción.
4.3.3	Publicación del build de la app	Generar y publicar la build de Flutter para pruebas.
4.3.4	Documentación OpenAPI/Swagger	Exponer y revisar la documentación automática de la API.
4.4	**Plan de monitoreo**	Observabilidad del sistema en producción.
4.4.1	Métricas de monitoreo	Definir indicadores: cobertura de colaboradores, errores de API, tiempos de respuesta.
4.4.2	Spring Boot Actuator + Logback	Configurar endpoints de salud/métricas y el registro histórico de logs.
4.4.3	UptimeRobot	Configurar el servicio gratuito que mantiene activo el backend.
4.5	**Plan de mantenimiento**	Sostenibilidad del sistema tras el piloto.
4.5.1	Tareas programadas (@Scheduled)	Eliminar automáticamente reportes GPS antiguos para mantener la base de datos liviana.
4.5.2	Respaldos automáticos (Neon)	Verificar las copias de seguridad automáticas del proveedor.
4.5.3	Actualización de rutas/paraderos	Documentar el procedimiento de mantenimiento evolutivo de la información.
4.6	**Entrega del Proyecto Final**	Cierre integral del proyecto.
4.6.1	Consolidación del informe final	Integrar todas las fases en el informe final del curso.
4.6.2	Ensayo y sustentación final	Preparar y ensayar la exposición final del proyecto.

Nota. Elaboración propia. EDT = estructura de desglose del trabajo; APF = avance de proyecto final.

Toma de requerimientos

Requerimientos funcionales

La Tabla 2 presenta los requerimientos funcionales del sistema (Orlando DJ Hernandez, s. f.).

Tabla 2

Requerimientos funcionales del sistema PiuraGo

Código	Descripción del requerimiento funcional
RF-01	El sistema debe permitir el registro y autenticación de usuarios mediante correo electrónico y contraseña (JWT).
RF-02	El sistema debe permitir consultar el listado de rutas de transporte urbano disponibles en Piura.

Código	Descripción del requerimiento funcional
RF-03	El sistema debe mostrar los paraderos asociados a una ruta seleccionada, en el orden secuencial correcto.
RF-04	El sistema debe permitir que el dispositivo del pasajero colaborador envíe periódicamente su ubicación GPS mientras viaja a bordo de una unidad.
RF-05	El sistema debe calcular y mostrar el tiempo estimado de llegada (ETA) de una unidad hacia un paradero específico, en base a los reportes GPS más recientes de los colaboradores.
RF-06	El sistema debe visualizar en un mapa interactivo la posición estimada de las unidades activas a partir del GPS compartido por colaboradores (polling cada 15–20 s).
RF-07	El sistema debe presentar un dashboard de "horas calientes" con las franjas horarias y tramos de mayor congestión por ruta, calculado sobre el histórico de reportes GPS.
RF-08	El sistema debe mostrar un indicador de cobertura (número de colaboradores activos) por ruta consultada.
RF-09	El sistema debe permitir enviar una notificación tipo geofencing cuando la unidad seleccionada esté próxima al paradero del usuario.

Nota. Elaboración propia. RF = requerimiento funcional.

Requerimientos no funcionales

La Tabla 3 presenta los requerimientos no funcionales del sistema.

Tabla 3

Requerimientos no funcionales del sistema PiuraGo

Código	Descripción del requerimiento no funcional
RNF-01	La actualización de posición de las unidades en el mapa no debe demorar más de 10 segundos respecto al ciclo de polling configurado.
RNF-02	El sistema debe seguir la arquitectura MVC + DAO aplicando principios SOLID, según lo exigido por el sílabo del curso.
RNF-03	Las contraseñas de usuario deben almacenarse cifradas (hash) y nunca en texto plano.
RNF-04	El backend debe exponer documentación de API generada automáticamente (OpenAPI/Swagger).
RNF-05	El sistema debe ser desplegable en un proveedor de hosting gratuito (Render/Railway) compatible con el stack elegido.
RNF-06	El código debe contar con pruebas automatizadas (unitarias y de integración) como parte del proceso de TDD exigido por el curso.
RNF-07	La interfaz debe ser usable en dispositivos móviles de gama media.
RNF-08	El sistema debe minimizar el tiempo de retención de la ubicación exacta de cada colaborador y evitar exponer su identidad a otros usuarios.

Nota. Elaboración propia. RNF = requerimiento no funcional.

Clasificación DINO / MoSCoW

Los requerimientos fueron priorizados según la técnica MoSCoW (Debe, Importante, Opcional, No van), lo que permite delimitar con claridad qué funcionalidades son indispensables para el piloto académico y cuáles quedan fuera del alcance de las 18 semanas del curso.

D — Debe (Must, obligatorio). RF-01, RF-02, RF-03, RF-04, RF-05, RF-06; RNF-02, RNF-03, RNF-06.

I — Importante (Should). RF-07, RF-08; RNF-01, RNF-04, RNF-05, RNF-07, RNF-08.

O — Opcional / Deseable (Could). RF-09 (notificación geofencing).

N — No van (Won't). N-01: posicionamiento GPS real de combis (hardware instalado por la empresa de transporte). N-02: comunicación en tiempo real verdadero vía WebSockets. N-03: verificación de identidad o validación de que el colaborador esté físicamente a bordo de la unidad. N-04: integración con sistemas de pago o venta de pasajes. N-05: gestión administrativa para las empresas de transporte.

Modelamiento de procesos de negocio – BPM

Importancia del modelamiento de procesos

El modelamiento de procesos permite ordenar las actividades de PiuraGo e identificar quién realiza cada una. En el proyecto intervienen pasajeros que comparten su ubicación mientras viajan y pasajeros que consultan información para planificar sus desplazamientos. Representar estas actividades facilita comprender cómo los reportes GPS llegan al sistema y se utilizan para ofrecer información de rutas, posiciones estimadas y tiempos de llegada.

El uso de BPMN permite mostrar la secuencia de actividades, las decisiones y la comunicación entre los participantes mediante una notación común. De esta manera, el equipo puede revisar la relación entre los requerimientos funcionales y el flujo de la aplicación, y detectar diferencias entre

la captura del GPS, el procesamiento de la información y su visualización. En PiuraGo, este modelamiento también permite representar la dependencia del servicio respecto a la colaboración de los pasajeros. Cuando no existen colaboradores activos en una ruta, la plataforma no puede ofrecer una posición actual ni un ETA basado en sus reportes. Asimismo, el proceso de colaboración contempla la decisión del pasajero de compartir su ubicación y la posibilidad de desactivar el envío.

Los diagramas sirven como referencia para el desarrollo y la revisión del sistema. Distinguen el envío de coordenadas cada 5–10 segundos de la actualización de la consulta mediante polling cada 15–20 segundos, y relacionan el histórico de reportes GPS con el dashboard de horas calientes.

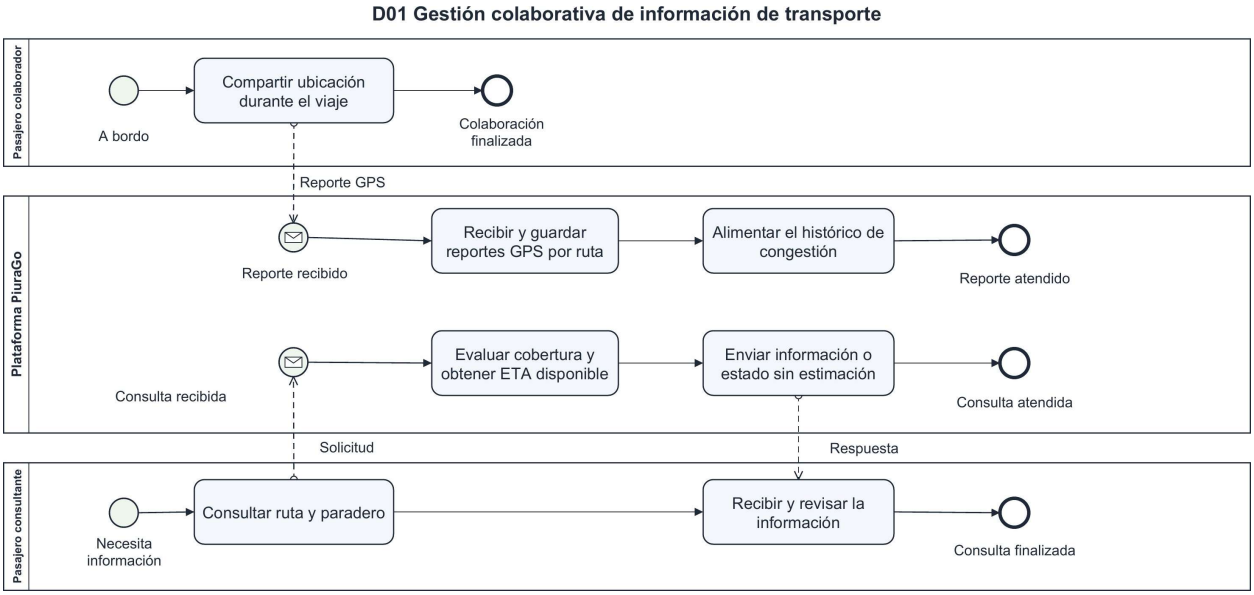
Diagramas BPMN del proyecto

Diagrama del proceso principal

Gestión colaborativa de información de transporte. Este diagrama tiene como objetivo representar cómo los reportes GPS de los pasajeros colaboradores permiten atender las consultas de rutas y ETA y alimentar el histórico de congestión. La Figura 4 presenta el diagrama.

Figura 4

Gestión colaborativa de información de transporte



Nota. Elaboración propia.

El pasajero colaborador comparte su ubicación durante el recorrido. PiuraGo recibe y guarda los reportes por ruta y alimenta el histórico. De manera independiente, el pasajero consultante selecciona una ruta y un paradero y solicita información. El sistema responde con la estimación disponible o con el estado sin colaboradores activos.

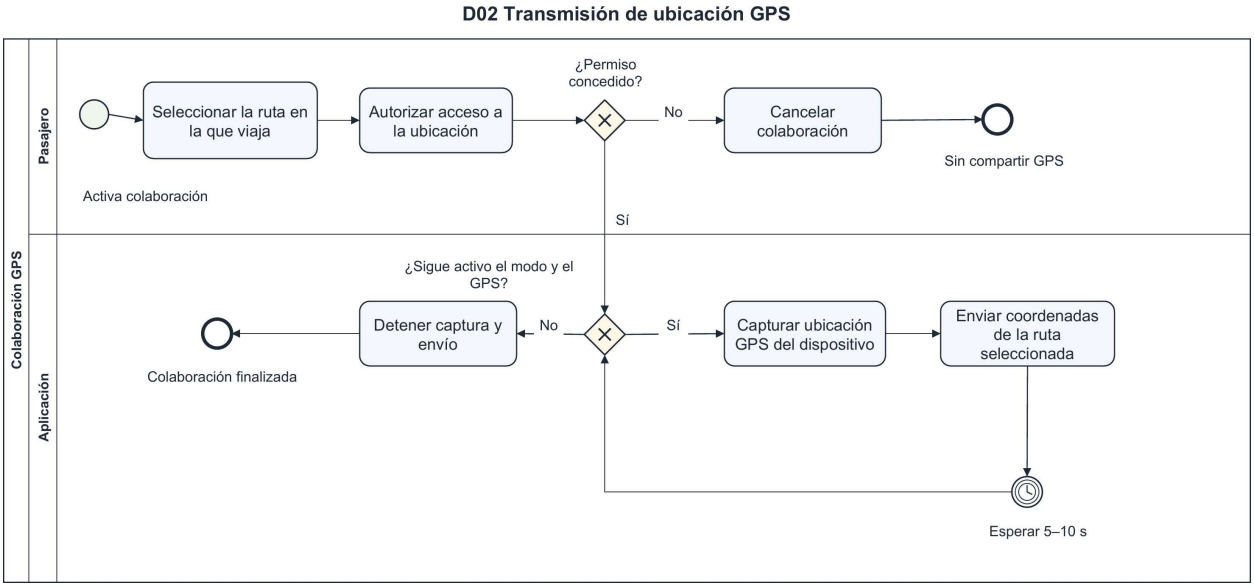
El proceso principal presenta una visión general. La consulta no obliga al pasajero consultante a compartir su GPS. El envío periódico y la consulta se detallan en los diagramas secundarios; los mensajes entre participantes no representan pasos de un único usuario.

Diagramas de procesos secundarios

Transmisión de ubicación GPS. Este diagrama tiene como objetivo representar la captura y el envío periódico de ubicación del pasajero colaborador. Requerimientos relacionados: RF-04 y RNF-08. La Figura 5 presenta el diagrama.

Figura 5

Transmisión de ubicación GPS



Nota. Elaboración propia.

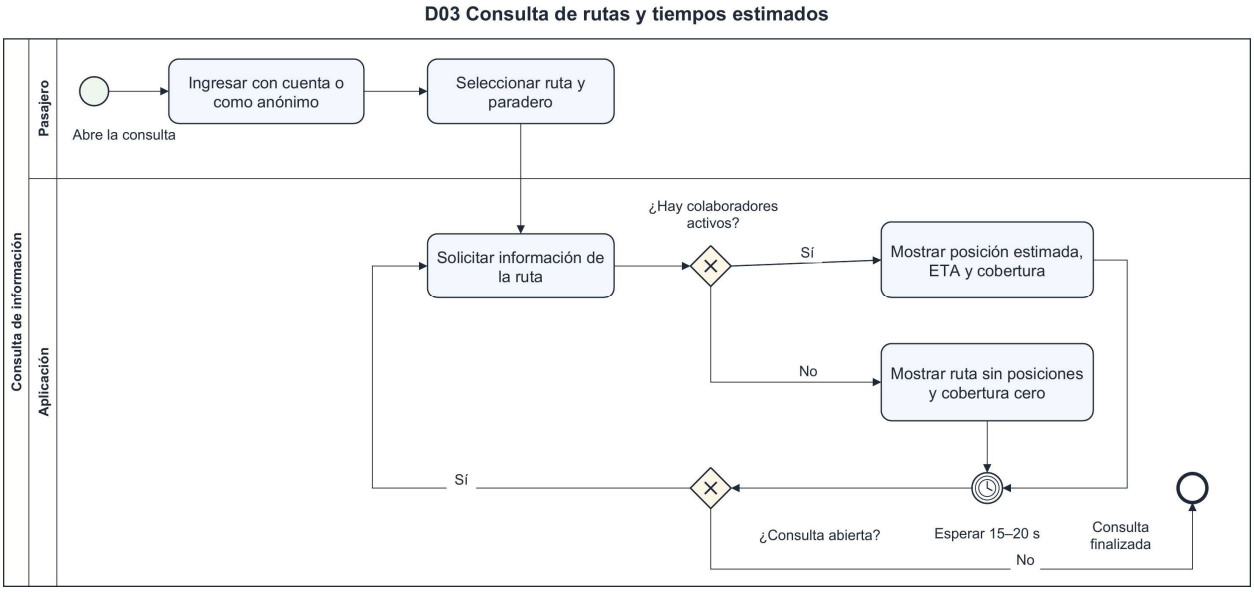
El pasajero selecciona la ruta en la que viaja y autoriza el acceso a la ubicación. Si no lo autoriza, la colaboración termina sin enviar coordenadas. Mientras el modo siga activo y el GPS esté disponible, la aplicación captura la ubicación, envía el reporte de la ruta seleccionada y espera entre 5 y 10 segundos antes de repetir el ciclo.

Al desactivar la colaboración se detienen la captura y el envío. Los reportes corresponden a la ubicación del dispositivo del pasajero; el proyecto no incluye la verificación de que el colaborador esté efectivamente a bordo de la unidad.

Consulta de rutas y tiempos estimados. Este diagrama tiene como objetivo mostrar al pasajero rutas, paraderos, posición estimada, ETA y cobertura de colaboradores. Requerimientos relacionados: RF-02, RF-03, RF-05, RF-06 y RF-08. La Figura 6 presenta el diagrama.

Figura 6

Consulta de rutas y tiempos estimados



Nota. Elaboración propia.

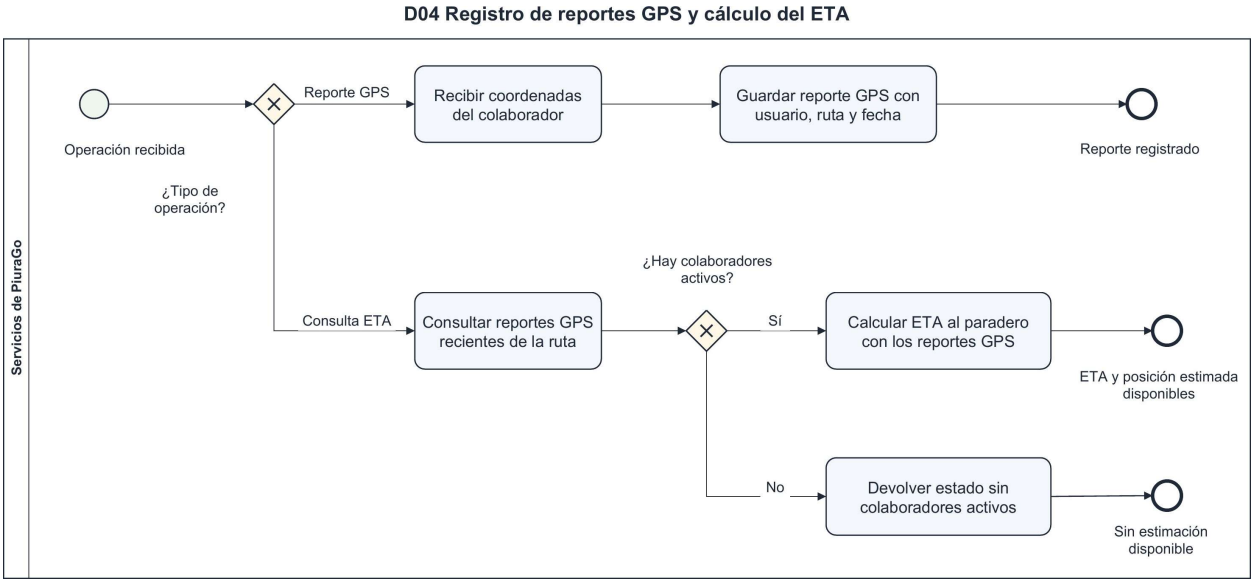
El pasajero ingresa con su cuenta o como anónimo y selecciona la ruta y el paradero. La aplicación solicita la información al sistema. Si hay colaboradores activos, presenta la posición estimada, el ETA y la cobertura. Si no hay colaboradores, mantiene la ruta visible e informa cobertura cero, sin posiciones actuales ni ETA disponible.

La consulta se actualiza mediante polling HTTP cada 15–20 segundos mientras la pantalla permanece abierta. Al cerrarla, no se realizan nuevas peticiones de actualización. La disponibilidad y precisión de las estimaciones dependen de los reportes de los colaboradores.

Registro de reportes GPS y cálculo del ETA. Este diagrama tiene como objetivo representar el almacenamiento de reportes y la atención de solicitudes de ETA. Requerimientos relacionados: RF-04, RF-05, RF-06 y RF-08. La Figura 7 presenta el diagrama.

Figura 7

Registro de reportes GPS y cálculo del ETA



Nota. Elaboración propia.

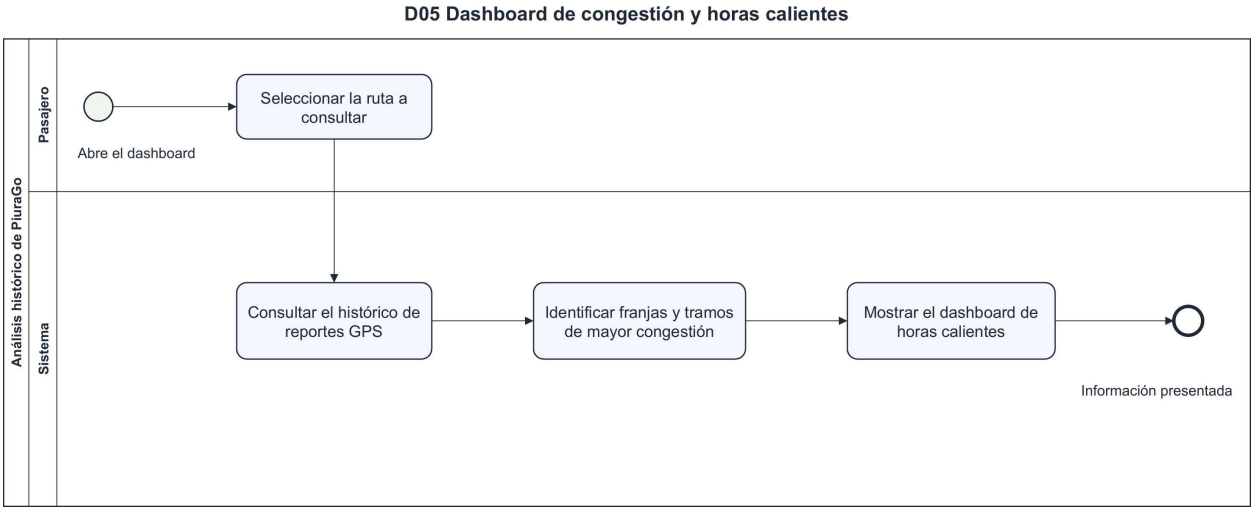
El sistema diferencia dos operaciones. Cuando recibe un reporte GPS, guarda las coordenadas asociadas al usuario, la ruta y la fecha y hora. Cuando recibe una consulta de ETA, obtiene los reportes recientes de la ruta y comprueba la disponibilidad de colaboradores activos antes de realizar el cálculo.

Con colaboradores activos, el sistema calcula el tiempo estimado de llegada al paradero seleccionado a partir de los reportes GPS. Sin colaboradores, devuelve el estado correspondiente y no presenta una estimación actual. El diagrama representa el cálculo a nivel funcional, de acuerdo con RF-05.

Dashboard de congestión y horas calientes. Este diagrama tiene como objetivo presentar las franjas horarias y los tramos con mayor congestión a partir del histórico GPS. Requerimiento relacionado: RF-07. La Figura 8 presenta el diagrama.

Figura 8

Dashboard de congestión y horas calientes



Nota. Elaboración propia.

El pasajero abre el dashboard y selecciona la ruta que desea consultar. El sistema recupera el histórico de reportes GPS, identifica las franjas horarias y los tramos de mayor congestión y presenta el resultado en el dashboard de horas calientes.

El análisis histórico permite al usuario identificar patrones de tráfico y ajustar sus horarios o rutas.

El dashboard presenta el análisis histórico de la ruta seleccionada.

Diseño de base de datos

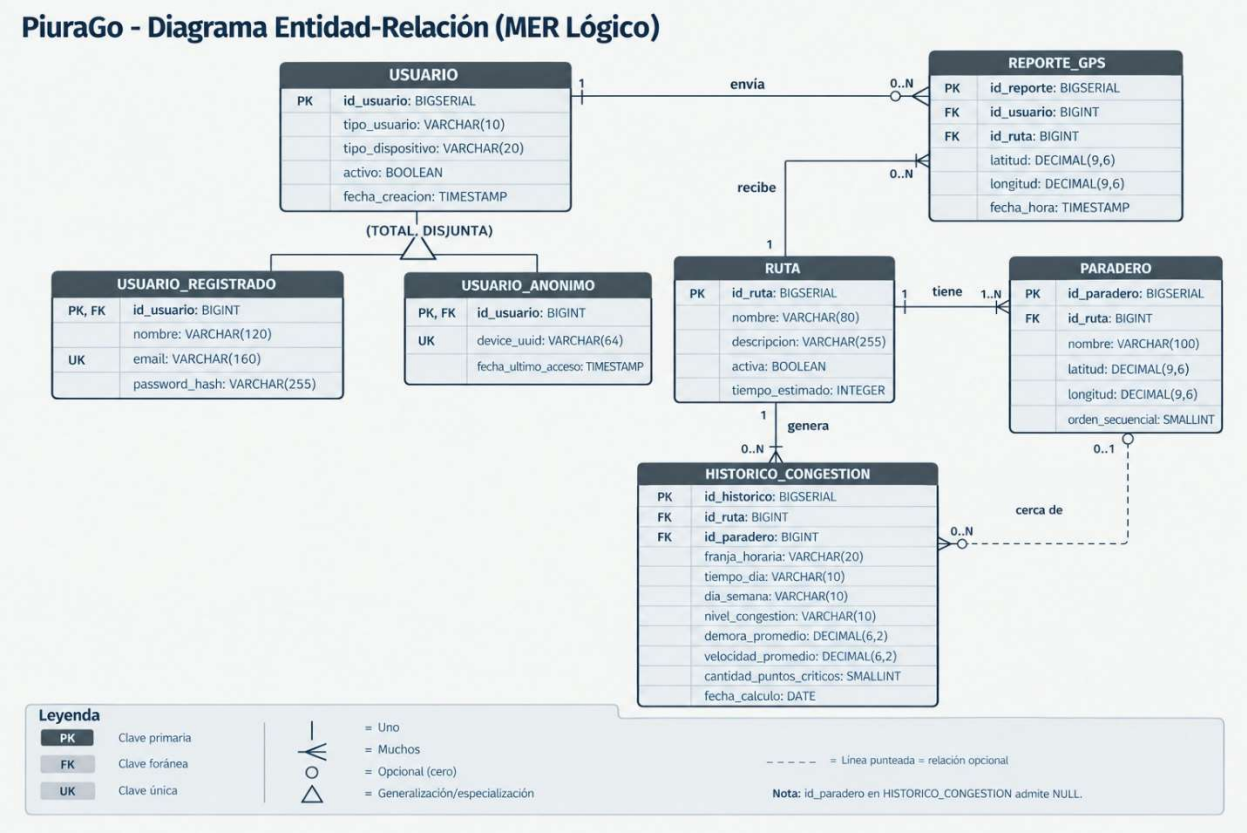
Diseño lógico de la base de datos

Modelo entidad-relación

La Figura 9 presenta el modelo entidad-relación (ER) de la base de datos del proyecto.

Figura 9

Modelo entidad-relación (ER) de la base de datos del proyecto



Nota. Elaboración propia.

Diseño físico de la base de datos

La Figura 10 presenta el script SQL de creación de las tablas e índices de la base de datos en PostgreSQL.

Figura 10

Script SQL de creación de la base de datos del proyecto

```
-- ------------------------------------------------------------------------
-- Tabla padre: USUARIO
-- Generalización TOTAL y DISJUNTA: todo usuario es ANONIMO o REGISTRADO,
-- nunca ambos, y no existe un usuario sin especializar.
-- ------------------------------------------------------------------------
CREATE TABLE usuario (
    id_usuario        BIGSERIAL PRIMARY KEY,
    tipo_usuario      VARCHAR(10) NOT NULL
                       CHECK (tipo_usuario IN ('ANONIMO', 'REGISTRADO')),
    tipo_dispositivo  VARCHAR(20),
    activo            BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_creacion    TIMESTAMP NOT NULL DEFAULT NOW()
);
-- ------------------------------------------------------------------------
-- Subclase: USUARIO_REGISTRADO
-- ------------------------------------------------------------------------
CREATE TABLE usuario_registrado (
    id_usuario     BIGINT PRIMARY KEY
                   REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    nombre         VARCHAR(120) NOT NULL,
    email          VARCHAR(160) NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL
);
-- ------------------------------------------------------------------------
-- Subclase: USUARIO_ANONIMO
-- ------------------------------------------------------------------------
CREATE TABLE usuario_anonimo (
    id_usuario            BIGINT PRIMARY KEY
                          REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    device_uuid           VARCHAR(64) NOT NULL UNIQUE,
    fecha_ultimo_acceso   TIMESTAMP NOT NULL DEFAULT NOW()
);
-- ------------------------------------------------------------------------
-- RUTA
-- ------------------------------------------------------------------------
CREATE TABLE ruta (
    id_ruta          BIGSERIAL PRIMARY KEY,
    nombre           VARCHAR(80) NOT NULL,
    descripcion      VARCHAR(255),
    activa           BOOLEAN NOT NULL DEFAULT TRUE,
    tiempo_estimado  INTEGER  -- minutos totales estimados del recorrido completo
);
-- ------------------------------------------------------------------------
-- PARADERO (1 ruta -> muchos paraderos)
-- ------------------------------------------------------------------------
CREATE TABLE paradero (
    id_paradero        BIGSERIAL PRIMARY KEY,
    id_ruta            BIGINT NOT NULL
                       REFERENCES ruta (id_ruta) ON DELETE CASCADE,
    nombre             VARCHAR(100) NOT NULL,
    latitud            DECIMAL(9,6) NOT NULL,
    longitud           DECIMAL(9,6) NOT NULL,
    orden_secuencial   SMALLINT NOT NULL,
    UNIQUE (id_ruta, orden_secuencial)
);
-- ------------------------------------------------------------------------
-- REPORTE_GPS (0 a muchos por usuario y por ruta)
-- ------------------------------------------------------------------------
```

```
CREATE TABLE reporte_gps (
    id_reporte   BIGSERIAL PRIMARY KEY,
    id_usuario   BIGINT NOT NULL
                 REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    id_ruta      BIGINT NOT NULL
                 REFERENCES ruta (id_ruta) ON DELETE CASCADE,
    latitud      DECIMAL(9,6) NOT NULL,
    longitud     DECIMAL(9,6) NOT NULL,
    fecha_hora   TIMESTAMP NOT NULL DEFAULT NOW()
);
-- Índice para las consultas de ETA (últimos reportes de una ruta, ordenados)
CREATE INDEX idx_reporte_gps_ruta_fecha
    ON reporte_gps (id_ruta, fecha_hora DESC);

-- Índice para consultar el historial de un usuario
CREATE INDEX idx_reporte_gps_usuario
    ON reporte_gps (id_usuario);
-- ------------------------------------------------------------------------
-- HISTORICO_CONGESTION
-- id_paradero es opcional: NULL = el registro aplica a toda la ruta;
-- con valor = el registro aplica a un tramo/paradero específico.
-- ------------------------------------------------------------------------
CREATE TABLE historico_congestion (
    id_historico             BIGSERIAL PRIMARY KEY,
    id_ruta                  BIGINT NOT NULL
                             REFERENCES ruta (id_ruta) ON DELETE CASCADE,
    id_paradero              BIGINT
                             REFERENCES paradero (id_paradero) ON DELETE SET NULL,
    franja_horaria           VARCHAR(20) NOT NULL,
    tiempo_dia               VARCHAR(10),
    dia_semana               VARCHAR(10) NOT NULL,
    nivel_congestion         VARCHAR(10) NOT NULL,
    demora_promedio          DECIMAL(6,2),
    velocidad_promedio       DECIMAL(6,2),
    cantidad_puntos_criticos SMALLINT,
    fecha_calculo            DATE NOT NULL DEFAULT CURRENT_DATE
);
-- Evita duplicar el mismo cálculo cuando el registro es de un paradero específico
CREATE UNIQUE INDEX ux_historico_con_paradero
    ON historico_congestion (id_ruta, id_paradero, franja_horaria, dia_semana)
    WHERE id_paradero IS NOT NULL;
-- Evita duplicar el mismo cálculo cuando el registro es a nivel de toda la ruta
CREATE UNIQUE INDEX ux_historico_sin_paradero
    ON historico_congestion (id_ruta, franja_horaria, dia_semana)
    WHERE id_paradero IS NULL;
-- Índice de apoyo para las consultas del dashboard de horas calientes
CREATE INDEX idx_historico_congestion_consulta
    ON historico_congestion (id_ruta, dia_semana, franja_horaria);
```

Nota. Elaboración propia. Código en lenguaje SQL para PostgreSQL.

Diseño UX/UI y prototipado

Identificación de usuarios y necesidades

A partir del Lean Canvas, el Project Charter y los requerimientos funcionales, se identificaron dos roles principales de usuario final y un rol técnico de soporte. La Tabla 4 resume cada rol con sus necesidades principales y los requerimientos asociados.

Tabla 4

Identificación de usuarios y necesidades del proyecto PiuraGo

Rol	Descripción	Necesidades principales	Requerimientos asociados
Pasajero Colaborador	Usuario a bordo de una combi/colectivo que activa el modo "Compartir GPS".	Activar/desactivar el envío de ubicación de forma simple, seleccionar la ruta en la que viaja, tener confianza en que su ubicación no se expone a terceros.	RF-01, RF-04, RNF-08
Pasajero Consultante	Usuario que espera en un paradero o planifica su desplazamiento.	Consultar rutas y paraderos, ver el ETA de la unidad, visualizar la posición estimada en un mapa, identificar horas/tramos de mayor congestión.	RF-02, RF-03, RF-05, RF-06, RF-07, RF-08, RF-09
Equipo de desarrollo (rol técnico)	Personas que revisan el sistema durante las entregas y sustentaciones.	Acceder a reportes de congestión exportables y a la documentación de la API.	RF-07, Apache POI

Nota. Elaboración propia.

Diseño UX

Diseño de las experiencias principales

Autenticación e inicio de sesión. Esta experiencia se define por lo siguiente:

- Flujo corto: correo y contraseña → emisión de JWT → redirección a la pantalla principal de la aplicación.
- Principio UX: mínimo número de campos y feedback inmediato de error.

Modo colaborador (compartir GPS). Esta experiencia se define por lo siguiente:

- El usuario selecciona la ruta en la que viaja.

- Activa el interruptor “Compartir ubicación”.
- La app captura GPS en segundo plano y envía coordenadas cada 5-10 segundos.
- Indicador visual claro de “estás colaborando” y posibilidad de desactivar en cualquier momento.
- Principio UX: control del usuario sobre su privacidad (RNF-08) y bajo consumo de batería.

Consulta de ruta. Esta experiencia se define por lo siguiente:

- Selección de ruta o paradero.
- Mapa interactivo con posición estimada de la unidad.
- Contador de ETA actualizado por polling cada 15-20 segundos.
- Indicador de cobertura (número de colaboradores activos).
- Principio UX: información prioritaria visible sin necesidad de desplazarse (ETA y cobertura en la parte superior).

Dashboard. Esta experiencia se define por lo siguiente:

- Visualización de franjas horarias y tramos con mayor congestión histórica.
- Filtro por ruta.
- Opción de exportar a Excel (Apache POI).
- Principio UX: lectura rápida de patrones (colores semáforo: verde/amarillo/rojo).

Diseño UI y prototipado

Prototipo de la solución

Se elaboró un prototipo funcional de alta fidelidad que cubre las pantallas principales del flujo BPM de PiuraGo:

- Login / Registro

- Selección de ruta
- Activación de GPS colaborativo
- Mapa interactivo con marcadores de unidad y paraderos
- Detalle de ETA
- Dashboard de congestión

Herramienta utilizada para el prototipado

Se utilizó Lovable como herramienta principal de prototipado.

Justificación de la elección:

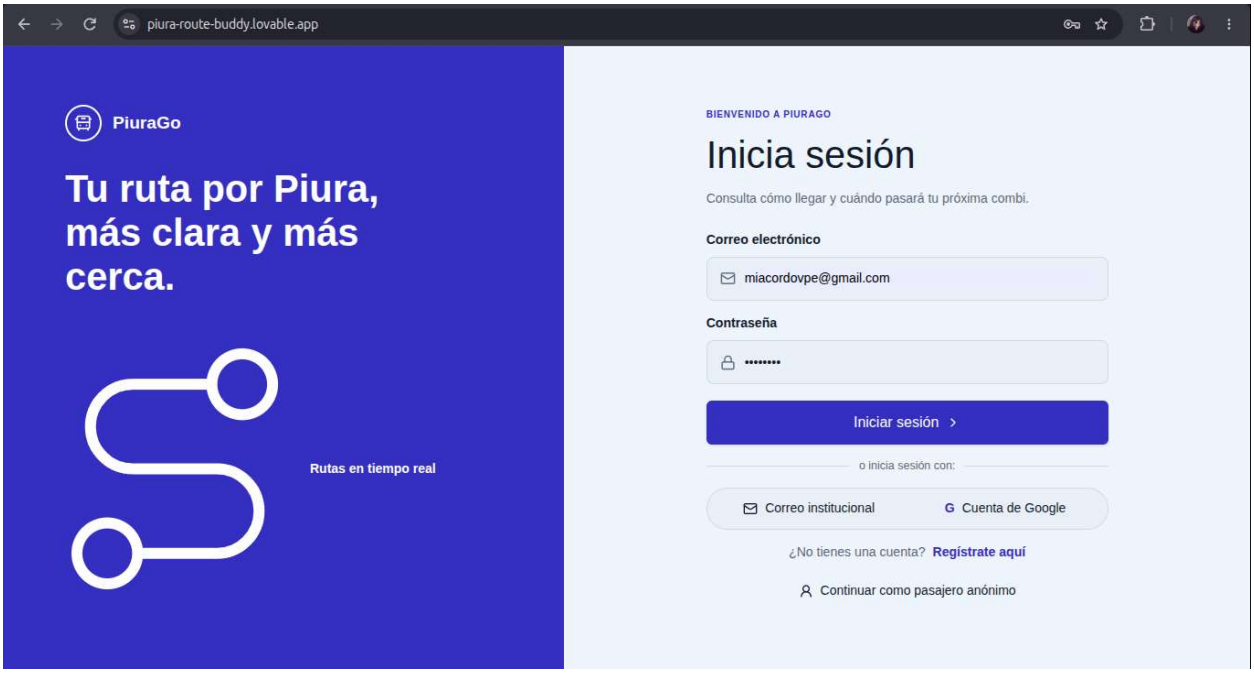
- Permite generar prototipos interactivos de alta fidelidad de forma rápida mediante prompts.
- Facilita la iteración del equipo sin necesidad de diseñar pantalla por pantalla desde cero.
- Genera interfaces modernas y responsivas, adecuadas para una aplicación multiplataforma.
- Se alinea con el enfoque ágil del proyecto y con la restricción de costo S/0.

Pantallas de los procesos principales

Inicio de sesión. Campos de correo y contraseña, botón para iniciar sesión, opciones de acceso con correo institucional o cuenta de Google, enlace para registrarse y opción para continuar como pasajero anónimo. La Figura 11 muestra esta pantalla.

Figura 11

Pantalla de inicio de sesión del prototipo de PiuraGo

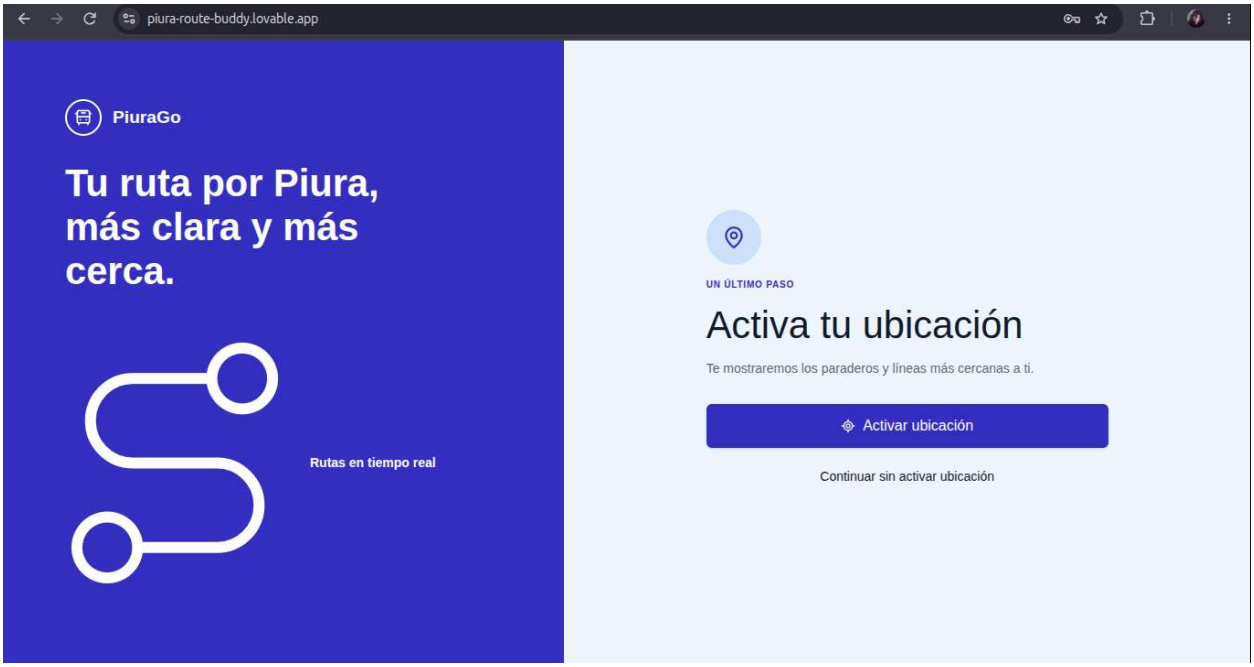


Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Activación de ubicación. Pantalla que solicita al usuario activar su ubicación para mostrar los paraderos y líneas más cercanas, con la opción de continuar sin activarla. La Figura 12 muestra esta pantalla.

Figura 12

Pantalla de activación de ubicación del prototipo de PiuraGo

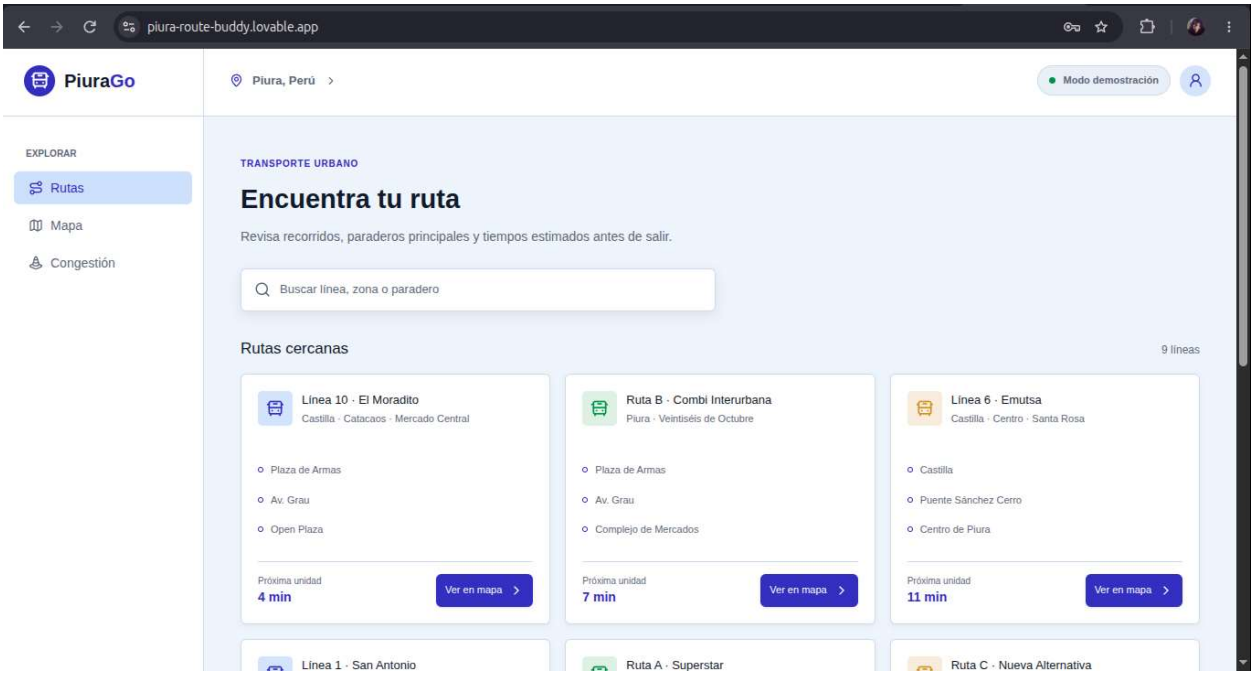


Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Rutas cercanas. Pantalla principal que presenta las líneas disponibles con sus paraderos principales y el tiempo de llegada de la próxima unidad, además de un buscador por línea, zona o paradero. La Figura 13 muestra esta pantalla.

Figura 13

Pantalla de rutas cercanas del prototipo de PiuraGo

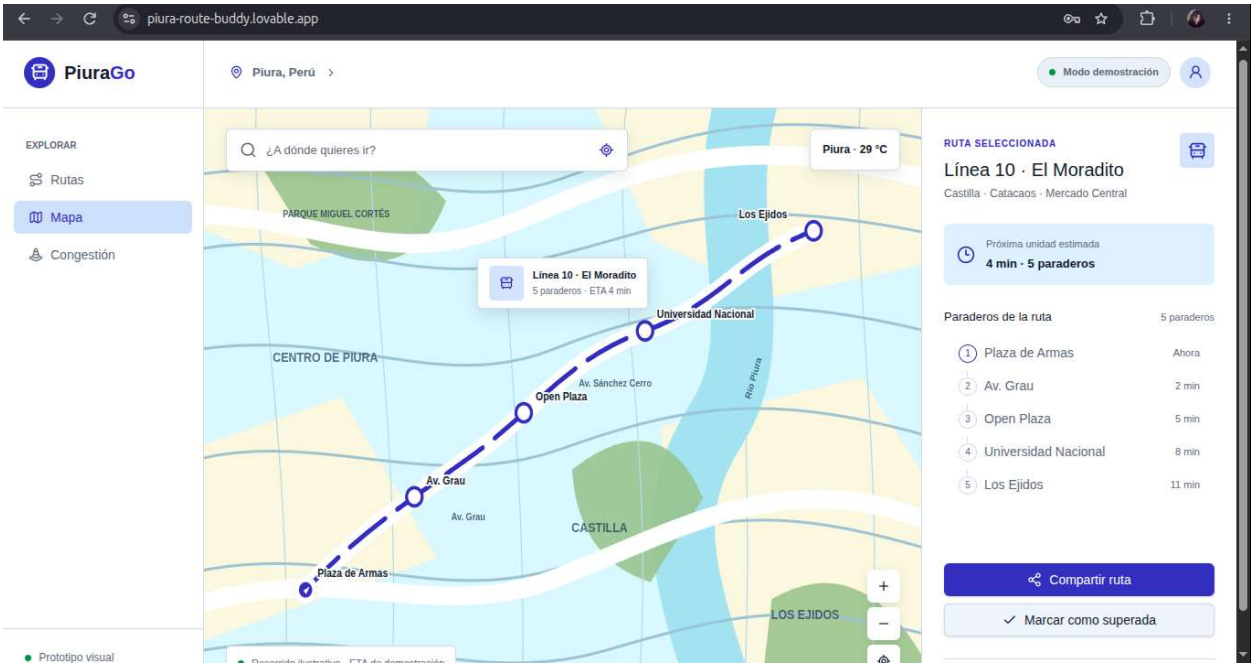


Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Mapa de la ruta seleccionada. Muestra el recorrido de la línea sobre el mapa con sus paraderos y el ETA estimado de cada uno, junto con las opciones de compartir la ruta y marcarla como superada. La Figura 14 muestra esta pantalla.

Figura 14

Pantalla del mapa de la ruta seleccionada del prototipo de PiuraGo

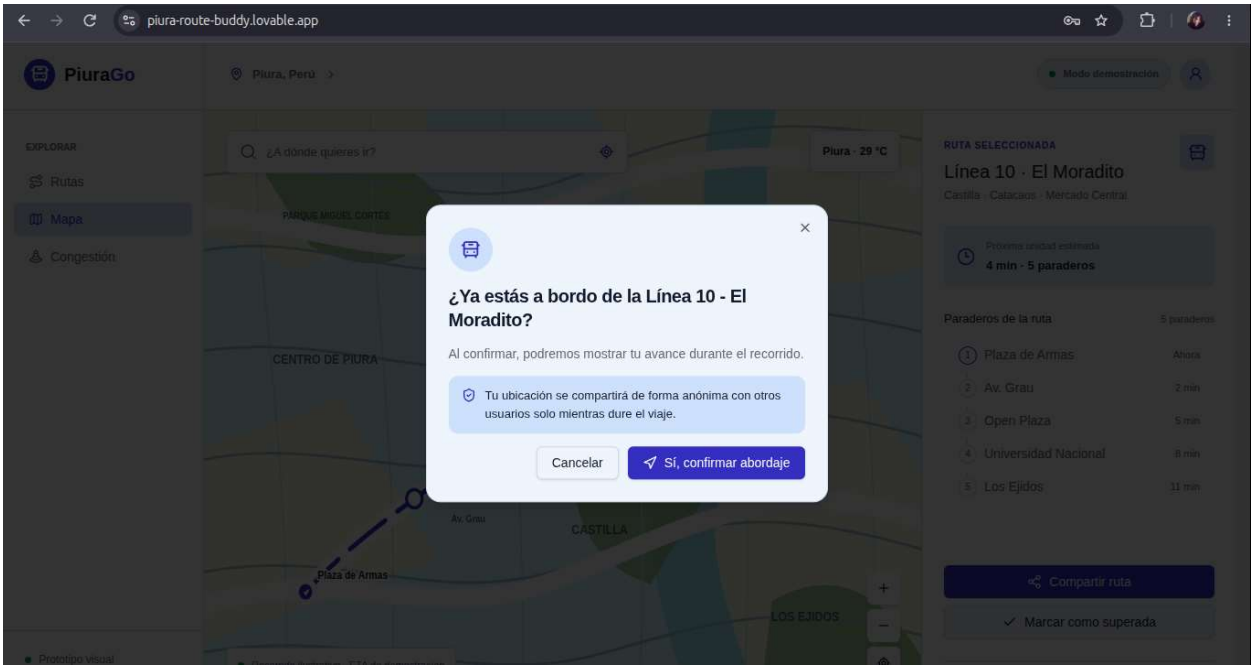


Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Confirmación de abordaje. Ventana que pregunta al pasajero si ya está a bordo de la unidad e informa que su ubicación se compartirá de forma anónima solo mientras dure el viaje. La Figura 15 muestra esta pantalla.

Figura 15

Pantalla de confirmación de abordaje del prototipo de PiuraGo

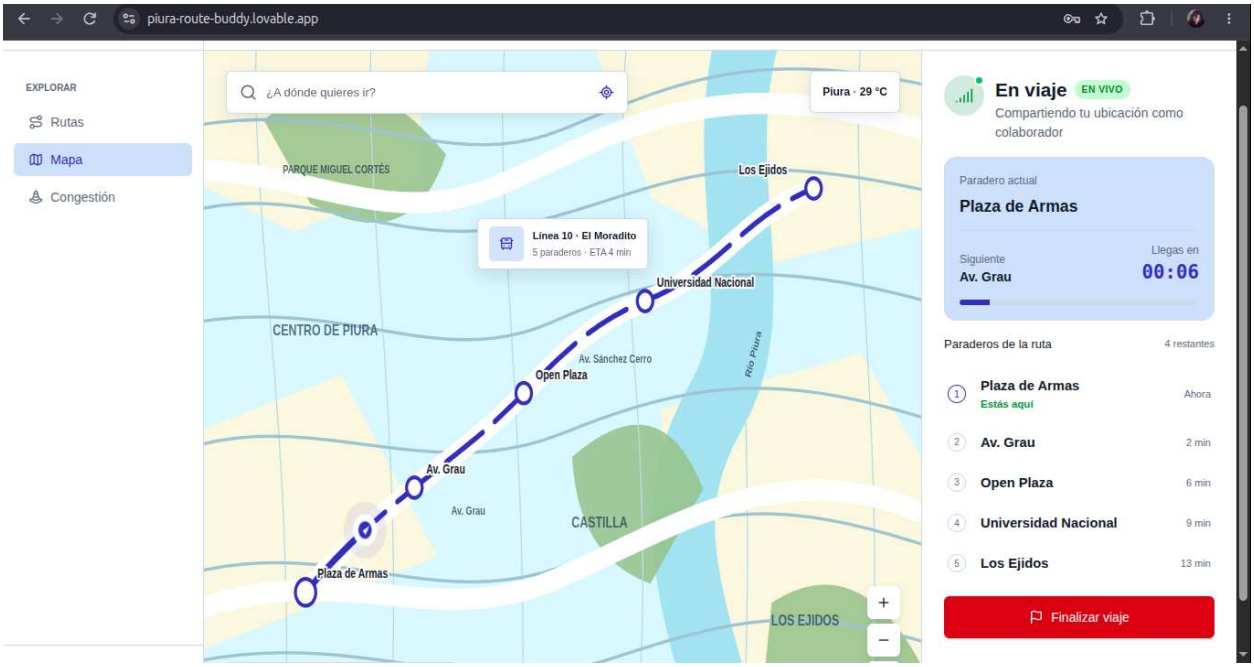


Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Viaje en curso. Vista en vivo que muestra el paradero actual, el siguiente paradero, el tiempo restante de llegada y los paraderos pendientes, mientras el pasajero comparte su ubicación como colaborador. La Figura 16 muestra esta pantalla.

Figura 16

Pantalla de viaje en curso del prototipo de PiuraGo

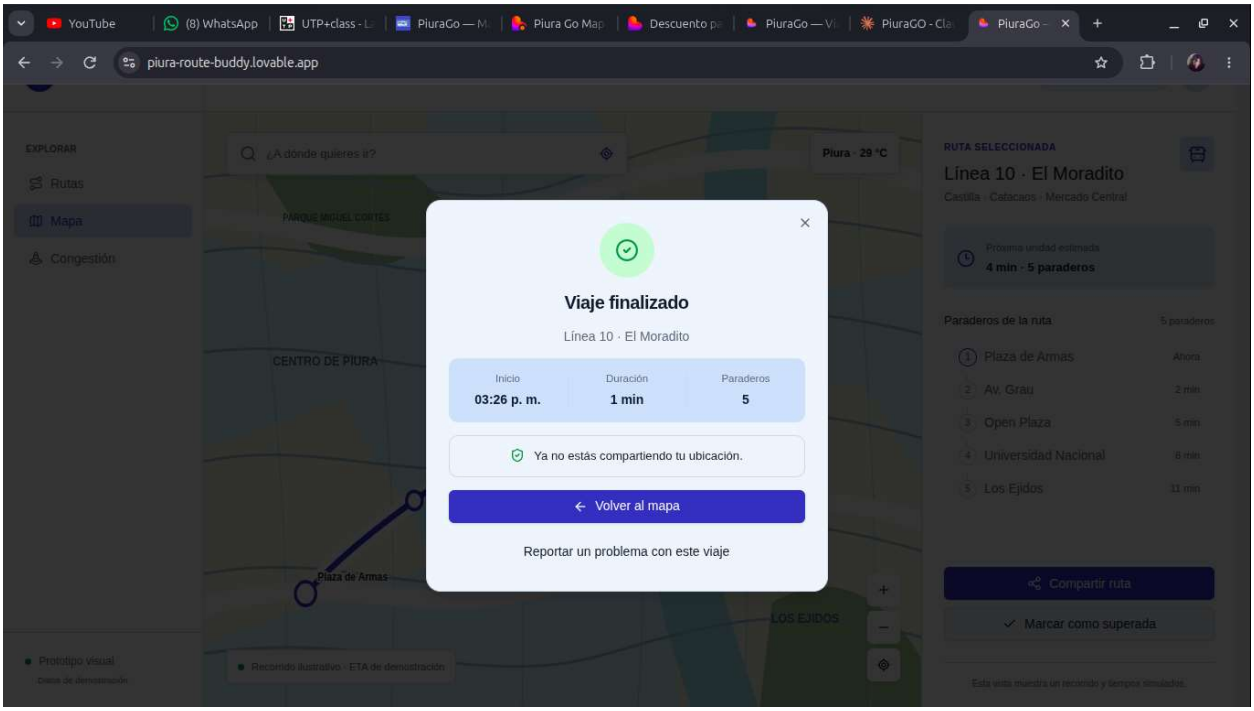


Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Viaje finalizado. Resumen del viaje con hora de inicio, duración y cantidad de paraderos, confirmación de que ya no se comparte la ubicación y opción para reportar un problema. La Figura 17 muestra esta pantalla.

Figura 17

Pantalla de viaje finalizado del prototipo de PiuraGo



Nota. Captura del prototipo de alta fidelidad elaborado en Lovable. Elaboración propia.

Reportes del sistema

Identificación de reportes

Reportes requeridos por el proyecto

La aplicación PiuraGo incluirá los siguientes reportes, diseñados para brindar información útil tanto a los usuarios finales como al equipo de desarrollo:

Dashboard. Muestra las franjas horarias y los tramos de ruta con mayor congestión. Se genera a partir del histórico de reportes GPS. Su objetivo es que el usuario pueda identificar los momentos y zonas de mayor tráfico para planificar mejor sus desplazamientos.

Cobertura de colaboradores por ruta. Indica cuántos pasajeros están compartiendo su ubicación en tiempo real en cada ruta. Permite al usuario saber si la información de ETA y posición es confiable en ese momento.

Histórico de ETA por paradero. Presenta la evolución del tiempo estimado de llegada en un paradero específico. Sirve para analizar patrones de demora y evaluar la precisión del cálculo del sistema.

Métricas de monitoreo del sistema. Incluye tiempos de respuesta, errores de la API y cobertura global de colaboradores. Se utiliza para verificar el correcto funcionamiento del backend durante las pruebas y el despliegue.

Referencias

Antonio Cedillo Hernandez. (s. f.). *Diagrama de Gantt* [Video]. YouTube. https://www.youtube.com/watch?v=xfxWFP_HxtE

Autor del canal. (s. f.). *Modelo de negocios: Lienzo Lean Canvas para proyectos de emprendimiento* [Video]. YouTube. https://youtu.be/GiuGihOWeBs

Cañabi Educa. (s. f.). *El proyecto y sus tipos* [Video]. YouTube. https://www.youtube.com/watch?v=rhAtUaDDtqA

Experiencias de un Maestro. (s. f.). *Gestión de proyectos - Project Charter* [Video]. YouTube. https://www.youtube.com/watch?v=QIyNutqnKXE

lepatru007. (s. f.). *Estructura de desglose de trabajo / Work Breakdown Structure EDT-WBS* [Video]. YouTube. https://www.youtube.com/watch?v=FiOZ6RgQQbc

Orlando DJ Hernandez. (s. f.). *Aprende qué son requerimientos de software* [Video]. YouTube. https://youtu.be/vD9YnZFynvg